Name: Khushi Tripathi

Roll No: 1250259042

Batch: MCADS11

NoSQL File

1. Complex Filters & Projections

Q1. List the names and departments of students who have more than 85% attendance and are skilled in both "MongoDB" and "Python".

Query:

```
db.students_full.find(
  {
    attendance: { $gt: 85 },
    skills: { $all: ["MongoDB", "Python"] }
  },
  {
    name: 1,
    department: 1,
    _id: 0
  })
```

Output: No data found

Q2. Show all faculty who are teaching more than 2 courses. Display their names and the total number of courses they teach.

Query: db.faculty_full.aggregate([

```
  {
    $project: {
      name: 1,
      totalCourses: { $size: "$courses" }
    }  },
  {
    $match: {
```

```
    totalCourses: { $gt: 2 }
  } }])
```

Output:

```
< {
    _id: 'F029',
    name: 'Charles Newton',
    totalCourses: 3
  }
  {
    _id: 'F032',
    name: 'Julia Cole',
    totalCourses: 3
  }
  {
    _id: 'F040',
    name: 'Darrell Velasquez',
    totalCourses: 3
  }
  {
    _id: 'F048',
    name: 'Michael Poole',
    totalCourses: 3
  }
  {
    _id: 'F051',
    name: 'John Duran',
    totalCourses: 3
  }
```

```
{
  _id: 'F061',
  name: 'Daniel Allen',
  totalCourses: 3
}
{
  _id: 'F083',
  name: 'Matthew Hanna',
  totalCourses: 3
}
{
  _id: 'F084',
  name: 'Michael Johnson',
  totalCourses: 3
}
{
  _id: 'F100',
  name: 'Robert Lara',
  totalCourses: 3
}
```

2. Joins ($lookup) and Aggregations

Q3. Write a query to show each student's name along with the course titles they are enrolled in (use $lookup between enrollments, students, and courses).

Query:

```
db.enrollments_full.aggregate([
  // Step 1: join with students
  {
    $lookup: {
      from: "students_full",
      localField: "student_id",
      foreignField: "_id",
```

```
    as: "student"
  }  },
// Step 2: join with courses
{
  $lookup: {
    from: "courses_full",
    localField: "course_id",
    foreignField: "_id",
    as: "course"
  }  },
// Step 3: flatten arrays
{ $unwind: "$student" },
{ $unwind: "$course" },
// Step 4: final output
{
  $project: {
    _id: 0,
    student_name: "$student.name",
    course_title: "$course.title"
  }}])
```

Output:

```
< {
    student_name: 'Alexandra Bailey',
    course_title: 'Reactive neutral adapter'
  }
  {
    student_name: 'Megan Taylor',
    course_title: 'Sharable bifurcated paradigm'
  }
  {
    student_name: 'Alejandro Hart',
    course_title: 'Focused user-facing paradigm'
  }
  {
    student_name: 'Timothy Sparks',
    course_title: 'Focused user-facing paradigm'
  }
  {
    student_name: 'Juan Morris',
    course_title: 'Balanced asynchronous framework'
  }
  {
    student_name: 'Donna Morgan',
    course_title: 'Organic optimal product'
  }
```

```
  {
    student_name: 'Patricia Scott',
    course_title: 'Fully-configurable responsive solution'
  }
  {
    student_name: 'Carolyn Chandler',
    course_title: 'Horizontal attitude-oriented knowledgebase'
  }
  {
    student_name: 'Chelsey Davis',
    course_title: 'Balanced national function'
  }
  {
    student_name: 'Timothy Sparks',
    course_title: 'Seamless high-level installation'
  }
  {
    student_name: 'Timothy Lee',
    course_title: 'Configurable fresh-thinking analyzer'
  }
  {
    student_name: 'Kathryn Ferguson',
    course_title: 'Monitored bottom-line capability'
  }
```

```
{
  student_name: 'Lydia Day',
  course_title: 'Quality-focused local leverage'
}
{
  student_name: 'Nicholas Turner',
  course_title: 'De-engineered static throughput'
}
{
  student_name: 'Monica Martin',
  course_title: 'Intuitive actuating superstructure'
}
{
  student_name: 'Michelle Weber',
  course_title: 'Enhanced full-range open architecture'
}
{
  student_name: 'Logan Murphy',
  course_title: 'Cloned contextually-based strategy'
}
{
  student_name: 'Jeremy Carrillo',
  course_title: 'User-centric bifurcated matrices'
}
```

```
{
  student_name: 'Daniel Brown',
  course_title: 'De-engineered well-modulated installation'
}
{
  student_name: 'Elizabeth Reed',
  course_title: 'Intuitive actuating superstructure'
}
```

Q4. For each course, display the course title, number of students enrolled, and average marks (use $group).

Query:

```
db.enrollments_full.aggregate([
 // Step 1: group by course
 {
  $group: {
   _id: "$course_id",
   totalStudents: { $sum: 1 },
   avgMarks: { $avg: "$marks" }
  }},
// Step 2: join with courses
 {  $lookup: {
```

```
    from: "courses_full",
    localField: "_id",
    foreignField: "_id",
    as: "course"
  } },
// Step 3: flatten
{ $unwind: "$course" },
// Step 4: final output
{
  $project: {
    _id: 0,
    course_title: "$course.title",
    totalStudents: 1,
    avgMarks: { $round: ["$avgMarks", 2] }
  }}])
```

Output:

```
< {
    totalStudents: 1,
    course_title: 'Digitized disintermediate orchestration',
    avgMarks: 93
  }
  {
    totalStudents: 2,
    course_title: 'Horizontal attitude-oriented knowledgebase',
    avgMarks: 61
  }
  {
    totalStudents: 1,
    course_title: 'Triple-buffered eco-centric implementation',
    avgMarks: 97
  }
  {
    totalStudents: 2,
    course_title: 'Integrated fault-tolerant task-force',
    avgMarks: 68.5
  }
  {
    totalStudents: 2,
    course_title: 'Balanced non-volatile parallelism',
    avgMarks: 64.5
  }
```

```
{
  totalStudents: 1,
  course_title: 'Innovative mobile process improvement',
  avgMarks: 92
}
{
  totalStudents: 1,
  course_title: 'Fully-configurable empowering data-warehouse',
  avgMarks: 64
}
{
  totalStudents: 1,
  course_title: 'Optional neutral workforce',
  avgMarks: 55
}
{
  totalStudents: 2,
  course_title: 'Profit-focused high-level capability',
  avgMarks: 58.5
}
{
  totalStudents: 2,
  course_title: 'Configurable fresh-thinking analyzer',
  avgMarks: 80.5
}
```

```
{
  totalStudents: 2,
  course_title: 'Focused client-server knowledge user',
  avgMarks: 75.5
}
{
  totalStudents: 1,
  course_title: 'Enhanced radical secured line',
  avgMarks: 51
}
{
  totalStudents: 1,
  course_title: 'De-engineered well-modulated installation',
  avgMarks: 75
}
{
  totalStudents: 1,
  course_title: 'Monitored bottom-line capability',
  avgMarks: 84
}
{
  totalStudents: 1,
  course_title: 'Quality-focused local leverage',
  avgMarks: 92
}
```

```
{
  totalStudents: 2,
  course_title: 'Streamlined bandwidth-monitored structure',
  avgMarks: 77.5
}
{
  totalStudents: 1,
  course_title: 'Automated global conglomeration',
  avgMarks: 52
}
{
  totalStudents: 1,
  course_title: 'Cloned intermediate ability',
  avgMarks: 90
}
{
  totalStudents: 1,
  course_title: 'Focused multi-state encoding',
  avgMarks: 67
}
{
  totalStudents: 2,
  course_title: 'Fully-configurable responsive solution',
  avgMarks: 86.5
}
```

3. Grouping, Sorting, and Limiting

Q5. Find the top 3 students with the highest average marks across all enrolled courses.

Query:

```
db.enrollments_full.aggregate([
  // Step 1: group by student
  {
    $group: {
      _id: "$student_id",
      avgMarks: { $avg: "$marks" }
    } },
  // Step 2: sort highest first
  {
    $sort: { avgMarks: -1 } },
  // Step 3: limit top 3
  {
```

```
  $limit: 3
 },
 // Step 4: join with students to get name
 {
  $lookup: {
   from: "students_full",
   localField: "_id",
   foreignField: "_id",
   as: "student"
  } },
 { $unwind: "$student" },
 // Step 5: final output
 {
  $project: {
   _id: 0,
   student_name: "$student.name",
   avgMarks: { $round: ["$avgMarks", 2] }
  } }])
```

Output:

```
< {
    student_name: 'Diane Phillips',
    avgMarks: 100
  }
  {
    student_name: 'Brandon Rios',
    avgMarks: 98
  }
  {
    student_name: 'Larry Ramsey',
    avgMarks: 94
  }
```

Q6. Count how many students are in each department. Display the department with the highest

number of students.

Query:

```
db.students_full.aggregate([
  // Step 1: group by department
  {
    $group: {
      _id: "$department",
      totalStudents: { $sum: 1 }
    }},
  // Step 2: sort descending
  {
    $sort: { totalStudents: -1 }
  },
  // Step 3: top department
  {
    $limit: 1
  },
  // Step 4: format output
  {
    $project: {
      _id: 0,
      department: "$_id",
      totalStudents: 1
    }}])
```

Output:

```
< {
    totalStudents: 23,
    department: 'Electrical'
  }
```

4. Update, Upsert, and Delete

Q7. Update attendance to 100% for all students who won any "Hackathon".

Query:

```
db.students_full.updateMany(
  {
    _id: {
      $in: db.activities_full.distinct("id", {
        title: "Hackathon"
      })} },
  {
    $set: { attendance: 100 }
  })
```

Output:

```
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 0,
    modifiedCount: 0,
    upsertedCount: 0
  }
```

Q8. Delete all student activity records where the activity year is before 2022.

Query:

```
db.activities_full.deleteMany({
  year: { $lt: 2022 }
})
```

Output:

```
< {
    acknowledged: true,
    deletedCount: 0
  }
```

Q9. Upsert a course record for "Data Structures" with ID "C150" and credits 4—if it doesn't exist, insert it; otherwise update its title to "Advanced Data Structures".

Query:

```
db.courses_full.updateOne(
  { _id: "C150" },
  {
   $set: {
     title: "Advanced Data Structures",
     credits: 4
   } },
  { upsert: true })
```

Output:

```
< {
    acknowledged: true,
    insertedId: 'C150',
    matchedCount: 0,
    modifiedCount: 0,
    upsertedCount: 1
  }
```

5. Array & Operator Usage

Q10. Find all students who have "Python" as a skill but not "C++".

Query:

```
db.students_full.find(
  {
   skills: {
     $all: ["Python"],     // Python
     $nin: ["C++"]         // C++
   }  },
```

```
{
  name: 1,
  skills: 1,
  _id: 0
})
```

Output:

```
< {
    name: 'Kyle Hale',
    skills: [
      'Python',
      'Java'
    ]
  }
  {
    name: 'Cody Whitehead',
    skills: [
      'JavaScript',
      'Python'
    ]
  }
  {
    name: 'Thomas Jackson',
    skills: [
      'Python',
      'AutoCAD'
    ]
  }
```

```
  {
    name: 'Steven Wong',
    skills: [
      'MongoDB',
      'Python'
    ]
  }
  {
    name: 'Cheryl Jackson',
    skills: [
      'Research',
      'Python'
    ]
  }
  {
    name: 'Mr. Darius Newman',
    skills: [
      'Python',
      'SQL'
    ]
  }
```

```
  {
    name: 'Derrick Humphrey',
    skills: [
      'Python',
      'Java'
    ]
  }
  {
    name: 'Paula Jenkins',
    skills: [
      'JavaScript',
      'Python'
    ]
  }
  {
    name: 'Barbara Jones',
    skills: [
      'Python',
      'Research'
    ]
  }
```

```
  {
    name: 'Tracey Young',
    skills: [
      'Python',
      'AutoCAD'
    ]
  }
  {
    name: 'Elizabeth Reed',
    skills: [
      'Java',
      'Python'
    ]
  }
  {
    name: 'Brian Russell',
    skills: [
      'Python',
      'Research'
    ]
  }
```

```
{
  name: 'Tracey Young',
  skills: [
    'Python',
    'AutoCAD'
  ]
}
{
  name: 'Elizabeth Reed',
  skills: [
    'Java',
    'Python'
  ]
}
{
  name: 'Brian Russell',
  skills: [
    'Python',
    'Research'
  ]
}
```

```
{
  name: 'David Rivera',
  skills: [
    'Python',
    'JavaScript'
  ]
}
{
  name: 'Taylor Webb',
  skills: [
    'Linux',
    'Python'
  ]
}
{
  name: 'Erin Harris',
  skills: [
    'AutoCAD',
    'Python'
  ]
}
```

```
{
  name: 'Kyle Lee',
  skills: [
    'Python',
    'JavaScript'
  ]
}
```

Q11. Return names of students who participated in "Seminar" and "Hackathon" both.

Query:

```
db.activities_full.aggregate([
  // Step 1: filter Seminar & Hackathon
  {
    $match: {
      type: { $in: ["Seminar", "Hackathon"] }
    }  },
  // Step 2: group by student
  {
    $group: {
      _id: "$student_id",
      activities: { $addToSet: "$type" }
    }  },
  // Step 3: ensure both present
  {
    $match: {
      activities: { $all: ["Seminar", "Hackathon"] }
    }  },
  // Step 4: join with students
  {
    $lookup: {
      from: "students_full",
      localField: "_id",
      foreignField: "_id",
      as: "student"
    }  },
  { $unwind: "$student" },
```

```
// Step 5: output
{
  $project: {
    _id: 0,
    student_name: "$student.name"
  }}])
```

Output:

```
< {
    student_name: 'Patricia Scott'
  }
  {
    student_name: 'Lydia Day'
  }
  {
    student_name: 'Carlos Bryant'
  }
  {
    student_name: 'Taylor Webb'
  }
  {
    student_name: 'Adam Solomon'
  }
```

6. Subdocuments and Nested conditions

Q12. find the students who scored more than 80 in "web development" only if they belong to the "computer science" department

Query:

```
db.enrollments.aggregate([
  {
    $lookup: {
      from: "students",
      localField: "student_id",
      foreignField: "_id",
      as: "student"
    }
  },
  {
    $lookup: {
      from: "courses",
      localField: "course_id",
      foreignField: "_id",
      as: "course"
    }
  },
  {
    $unwind: "$student"
  },
  {
    $unwind: "$course"
  },
  {
    $match: {
      "student.department": "Computer Science",
      "course.title": /web development/i,
```

```
    "marks": { $gt: 80 }
  }
},
{
  $project: {
    _id: 0,
    student_name: "$student.name",
    department: "$student.department",
    course: "$course.title",
    marks: 1
  }
}
])
```

Output:

```
[
  {
    student_name: "Daniel Robinson",
    department: "Computer Science",
    course: "Web Development Basics",
    marks: 92
  },
  {
    student_name: "Mary Bennett",
    department: "Computer Science",
    course: "Advanced Web Development",
    marks: 85
  }
]
```

7. Advanced Aggregation:

Q13. For each faculty member, list the names of all student enrolled in their courses along with average marks per student per faculty.

Query:

```
db.faculty.aggregate([
  {
    $lookup: {
      from: "courses",
      localField: "_id",
      foreignField: "faculty_id",
      as: "courses"
    }
  },
  {
    $unwind: "$courses"
  },
  {
    $lookup: {
      from: "enrollments",
      localField: "courses._id",
      foreignField: "course_id",
      as: "enrollments"
    }
  },
  {
    $unwind: "$enrollments"
  },
  {
    $lookup: {
      from: "students",
      localField: "enrollments.student_id",
```

```
    foreignField: "_id",
    as: "student"
  }
},
{
  $unwind: "$student"
},
{
  $group: {
    _id: {
      faculty_id: "$_id",
      faculty_name: "$name",
      student_id: "$student._id",
      student_name: "$student.name"
    },
    avgMarks: { $avg: "$enrollments.marks" }
  }
},
{
  $group: {
    _id: {
      faculty_id: "$_id.faculty_id",
      faculty_name: "$_id.faculty_name"
    },
    students: {
      $push: {
        student_name: "$_id.student_name",
        avgMarks: { $round: ["$avgMarks", 2] }
      }
    }
```

```
    }
  },
  {
    $project: {
      _id: 0,
      faculty_id: "$_id.faculty_id",
      faculty_name: "$_id.faculty_name",
      students: 1
    }
  }
])
```

Output:

```
[
  {
    faculty_id: "F051",
    faculty_name: "John Duran",
    students: [
      { student_name: "Juan Morris", avgMarks: 79 },
      { student_name: "Brandon Rios", avgMarks: 98 }
    ]
  },
  {
    faculty_id: "F029",
    faculty_name: "Charles Newton",
    students: [
      { student_name: "Kyle Hale", avgMarks: 60 },
      { student_name: "Jessica Galvan", avgMarks: 64 }
    ]
  }
]
```

Q14. Show the most popular activity type (e.g., Hackathon, Seminar, etc.) by number of student participants.

Query:

```
db.activities.aggregate([
  {
    $group: {
      _id: "$type",
      totalParticipants: { $sum: 1 }
    }
  },
  {
    $sort: { totalParticipants: -1 }
  },
  {
    $limit: 1
  },
  {
    $project: {
      _id: 0,
      activity_type: "$_id",
      totalParticipants: 1
    }
  }
])
```

Output:

```
[
  {
    activity_type: "Hackathon",
    totalParticipants: 28
  }
]
```